

Project 1

Teresa Macarulla Uriarte
(Dated: September 8, 2025)

<https://github.com/Teresa-Macarulla/Comp.Phy.Projects/tree/master/P1>

PROBLEM 1

The one-dimensional Poisson equation can be written as:

$$-\frac{d^2u}{dx^2} = f(x) \quad (1)$$

Here de $f(x)$ function is:

$$f(x) = 100 e^{-10x} \quad (2)$$

To check analytically that an exact solution to Eq. 1 is given by

$$u(x) = 1 - (1 - e^{-10})x - e^{-10x}, \quad (3)$$

we have to differentiate $u(x)$ with respect to x twice:

$$u'(x) = -(1 - e^{-10}) + 10 e^{-10x} \Rightarrow u''(x) = -100 e^{-10x} \Rightarrow -u''(x) = f(x) \quad (4)$$

PROBLEM 2

Eq. 3 is plotted in Fig. 1 in the interval $x \in [0, 1]$ which is defined by the boundary $u(0) = 0$ and $u(1) = 0$.

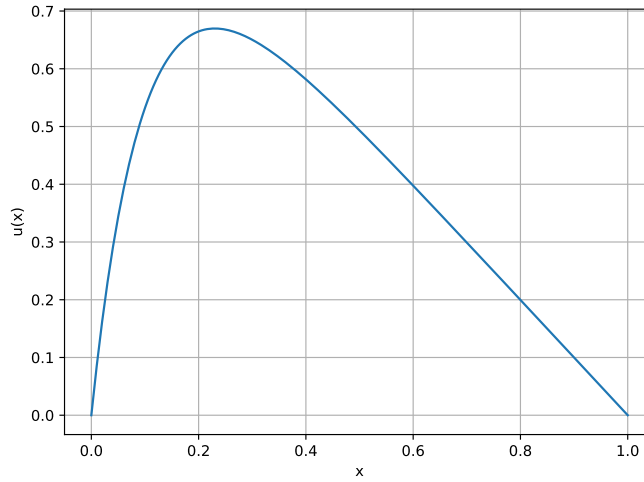


FIG. 1. Exact solution $u(x)$ (Eq. 3) represented in the domain $x \in [0, 1]$. The boundary conditions are satisfied.

Fig. 1 was created from the Python script `plot.py` and using the data in `data.txt` in the GitHub repository. The data points in `data.txt` were generated by the C++ program in `problem2.cpp`, also in the GitHub repository.

PROBLEM 3

To derive a discretize the Poisson equation (Eq. 1), we start by adding the Taylor expansions for $u(x + h)$ and $u(x - h)$:

$$u(x + h) = u(x) + u'(x)h + \frac{1}{2}u''(x)h^2 + \frac{1}{6}u'''(x)h^3 + \mathcal{O}(h^4) \quad (5)$$

$$u(x - h) = u(x) - u'(x)h + \frac{1}{2}u''(x)h^2 - \frac{1}{6}u'''(x)h^3 + \mathcal{O}(h^4). \quad (6)$$

$$u(x + h) + u(x - h) = 2u(x) + u''(x)h^2 + \mathcal{O}(h^4) \quad (7)$$

Isolating $u''(x)$ from Eq. 7 and substituting in Eq. 1, we obtain the derivative version of the Poisson equation:

$$\frac{-u(x + h) + 2u(x) - u(x - h)}{h^2} + \mathcal{O}(h^2) = f(x) \quad (8)$$

The discretized version of the Poisson equation is:

$$\frac{-v_{i+1} + 2v_i - v_{i-1}}{h^2} = f_i, \quad (9)$$

where v_i represents an approximation to $u(x)$ and $f_i \equiv f(x)$.

PROBLEM 4

We start from the centered finite difference discretization, which for each interior point i yields

$$-v_{i-1} + 2v_i - v_{i+1} = h^2 f_i. \quad (10)$$

Notice that (10) actually represents a *set* of equations, one for each interior index i . Our goal is therefore to determine the unknowns

$$v_1, v_2, \dots, v_{n-1},$$

given that the boundary values v_0 and v_n are known, as well as all the source terms f_i .

To illustrate the procedure, let us consider the particular case $n_{\text{steps}} = 5$, which gives six nodes:

$$v_0, v_1, v_2, v_3, v_4, v_5.$$

Here, v_0 and v_5 are boundary conditions, so the unknowns to be solved for are

$$v_1, v_2, v_3, v_4.$$

The corresponding four equations can be written explicitly as

$$\begin{aligned} -v_0 + 2v_1 - v_2 &= h^2 f_1, \\ -v_1 + 2v_2 - v_3 &= h^2 f_2, \\ -v_2 + 2v_3 - v_4 &= h^2 f_3, \\ -v_3 + 2v_4 - v_5 &= h^2 f_4. \end{aligned}$$

Since v_0 and v_5 are known, we can define

$$\begin{aligned} g_1 &\equiv h^2 f_1 + v_0, \\ g_2 &\equiv h^2 f_2, \\ g_3 &\equiv h^2 f_3, \\ g_4 &\equiv h^2 f_4 + v_5. \end{aligned}$$

With this notation, the system takes the matrix form

$$\begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix} = \begin{bmatrix} g_1 \\ g_2 \\ g_3 \\ g_4 \end{bmatrix}.$$

Here we obtain a matrix equation of the form:

$$\mathbf{A}\vec{v} = \vec{g}. \quad (11)$$

PROBLEM 5

Problem a

Let the complete discrete solution be

$$\vec{v}^* \in \mathbb{R}^m, \quad \vec{v}^* = \begin{bmatrix} v_0 \\ v_1 \\ \vdots \\ v_{m-2} \\ v_{m-1} \end{bmatrix},$$

where v_0 and v_{m-1} are the boundary values and $v_1, \dots, v_{m-2}$ are interior unknowns. The linear system is assembled only for the interior entries, hence the matrix A is $n \times n$ with

$$n = m - 2 \quad (\text{equivalently } m = n + 2).$$

Problem b

Solving

$$A\vec{v} = \vec{g}$$

returns the interior part of the complete solution,

$$\vec{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_{m-3} \\ v_{m-2} \end{bmatrix} \in \mathbb{R}^n,$$

while the boundary components v_0 and v_{m-1} are provided by the boundary conditions (not by the solve).

PROBLEM 6

Problem a

Let's consider the Thomas algorithm (Gaussian elimination on a tridiagonal matrix) to solve $\mathbf{A}\vec{v} = \vec{g}$. Let $\mathbf{A}$ be a general tridiagonal matrix, with vectors $\vec{a}$, $\vec{b}$ and $\vec{c}$ representing the subdiagonal, main diagonal and superdiagonal, respectively.

Algorithm 1 general algorithm

Input: subdiagonal $\vec{a} = [a_2, a_3, \dots, a_n]$, diagonal $\vec{b} = [b_1, b_2, \dots, b_n]$, superdiagonal $\vec{c} = [c_1, c_2, \dots, c_{n-1}]$, right-hand side $\vec{g} = [g_1, g_2, \dots, g_n]$

Output: solution $\vec{v} = [v_1, v_2, \dots, v_n]$ to $A\vec{v} = \vec{g}$

Forward substitution

$\tilde{b}_1 \leftarrow b_1, \quad \tilde{g}_1 \leftarrow g_1$

for $i = 2, \dots, n$ **do**

$$m \leftarrow \frac{a_i}{\tilde{b}_{i-1}}$$

$$\tilde{b}_i \leftarrow b_i - m c_{i-1}$$

$$\tilde{g}_i \leftarrow g_i - m \tilde{g}_{i-1}$$

Back substitution

$$v_n \leftarrow \frac{\tilde{g}_n}{\tilde{b}_n}$$

for $i = n - 1, \dots, 1$ **do**

$$v_i \leftarrow \frac{\tilde{g}_i - c_i v_{i+1}}{\tilde{b}_i}$$

Problem b

A floating-point operation (FLOP) is a single arithmetic operation performed on floating-point numbers, typically counted as one of: addition, subtraction, multiplication, or division.

We can add the operations required for the Thomas algorithm in the Forward substitution and Back substitution:

Forward substitution (for $i = 2, \dots, n$):

$$m = \frac{a_i}{\tilde{b}_{i-1}} \text{ (1 div), } \quad \tilde{b}_i = b_i - m c_{i-1} \text{ (1 mult + 1 sub), } \quad \tilde{g}_i = g_i - m \tilde{g}_{i-1} \text{ (1 mult + 1 sub),}$$

We have 5 FLOPs per step, that is, $5(n - 1)$ FLOPs for the Forward substitution.

Back substitution:

$$v_n = \frac{\tilde{g}_n}{\tilde{b}_n} \text{ (1 div), } \quad v_i = \frac{\tilde{g}_i - c_i v_{i+1}}{\tilde{b}_i} \text{ (1 mult + 1 sub + 1 div)}$$

We have 3 FLOPs per step and the division before the loop, that is, $3(n - 1) + 1$ FLOPs for the Back substitution.

Total: FLOPs = $5(n - 1) + [3(n - 1) + 1] = 8n - 7$.

PROBLEM 7

Problem a

In order to solve the discretised Poisson equation in the form

$$A\vec{v} = \vec{g},$$

where A is the tridiagonal matrix introduced in Problem 4, we implemented a C++ program named `problem7.cpp`, which is available in the GitHub repository. This program applies the *general algorithm* described in Problem 6a) to efficiently solve the tridiagonal linear system. The algorithm is based on Gaussian elimination adapted to the tridiagonal case (Thomas algorithm).

The program not only computes the numerical solution $\vec{v}$ for a given discretization, but also stores both $\vec{v}$ and the corresponding grid points $\vec{x}$ in an output text file called `data2.txt`, which is likewise available in the repository. This output is then used for visualization and comparison with the exact solution.

Problem b

To visualize and compare the numerical results with the analytical solution $u(x)$ given in Eq. 3, we developed a Python script named `plot2.py`, also included in the GitHub repository. This script reads the data from `data2.txt` and generates the figure `plot2.pdf`, which is shown below.

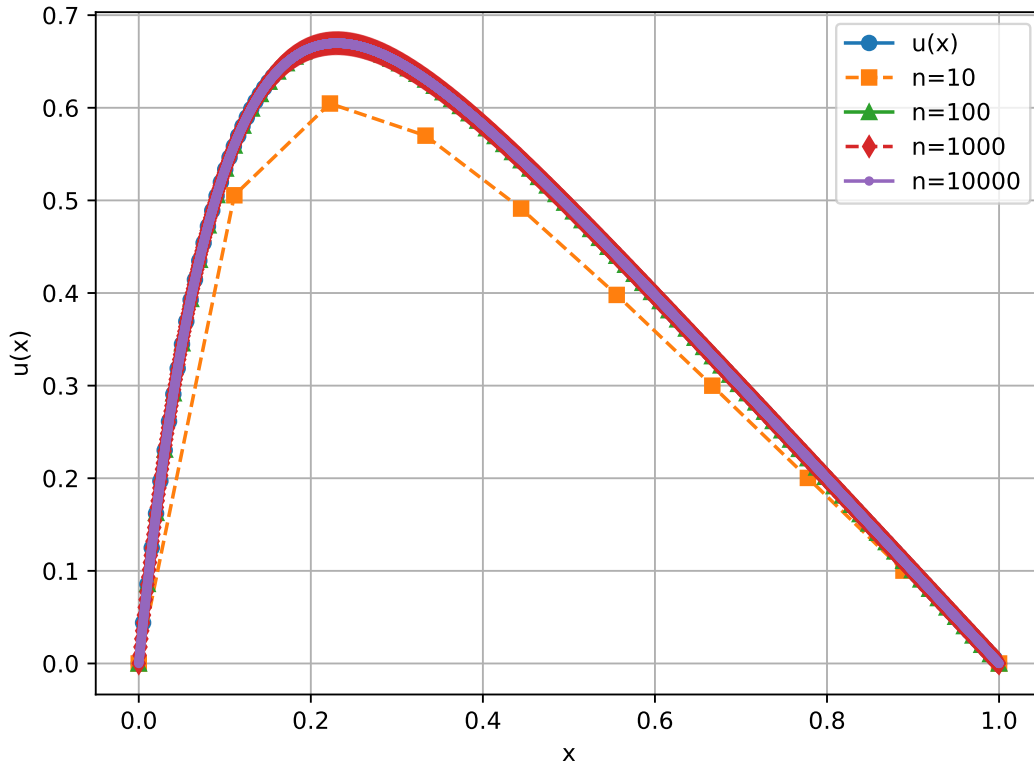


FIG. 2. Comparison between the exact solution $u(x)$ and the numerical solutions obtained with different discretizations ($n = 10, 100, 1000, 10000$ points).

The results in Fig. 2 clearly show that for the discretisation $n = 10$ the numerical solution deviates significantly from the analytical curve. However, starting from $n = 100$ grid points and above, the discrete solutions rapidly converge and align very closely with the exact solution $u(x)$. This confirms the consistency of the discretisation scheme and the reliability of the tridiagonal solver for large system sizes.

PROBLEM 8

Problem a

We can compute the logarithm of the absolute error of the approximated values v_i obtained by implementing the *general algorithm* using Eq. 12:

$$\log_{10}(\Delta_i) = \log_{10}(|u_i - v_i|), \quad (12)$$

where the values u_i correspond to the exact solution $u(x)$ given in Eq. 3, evaluated at the discrete points x_i .

The file `problem8.cpp`, available in the GitHub repository, contains a C++ program that reads the data pairs (x_i, v_i) from `data2.txt` and computes (the logarithm of) the absolute error between the numerical approximation v_i and the analytical solution $u(x)$ for the discretization cases $n = 10, 100, 1000$, and 10000 .

The logarithm of the absolute errors corresponding to each discretisation point are then written to an output file named `data3_abs.txt`, which is also included in the repository. Fig. 3 shown below presents the logarithm of the absolute errors for all discretization cases and was generated using the Python script `plot3.py`, likewise provided in the GitHub repository.

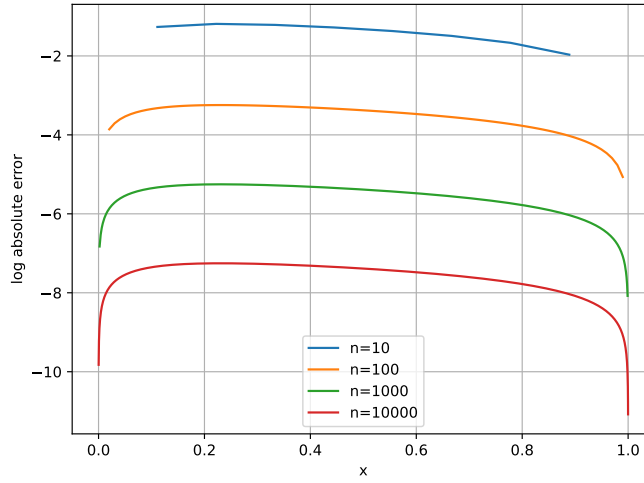


FIG. 3. Logarithm of the absolute error representation (Eq. 12) for the numerical approximations v_i with respect to the exact solution $u(x)$, obtained for discretizations with $n = 10, 100, 1000$, and 10000 points.

Note that for a finer discretisation (larger n) the absolute error is reduced. Moreover, the error at the boundaries of the interval $[0, 1]$ reduces rapidly, as expected. The boundary values have not been included in Fig. 3.

Problem b

We can compute (the logarithm of) the relative error of the approximated values v_i obtained by implementing the *general algorithm* using Eq. 13:

$$\log_{10}(\epsilon_i) = \log_{10} \left(\left| \frac{u_i - v_i}{u_i} \right| \right), \quad (13)$$

where the values u_i correspond to the exact solution $u(x)$ given in Eq. 3, evaluated at the discrete points x_i .

The file `problem8.cpp`, available in the GitHub repository, contains a C++ program that reads the data pairs (x_i, v_i) from `data2.txt` and computes the logarithm of the relative error between the numerical approximation v_i and the analytical solution $u(x)$ for the discretisation cases $n = 10, 100, 1000$, and 10000 .

The logarithm of the relative errors corresponding to each discretisation point are then written to an output file named `data3_rel.txt`, which is also included in the repository. Fig. 4 shown below presents the logarithm of the relative errors for all discretisation cases and was generated using the Python script `plot3.py`, likewise provided in the GitHub repository.

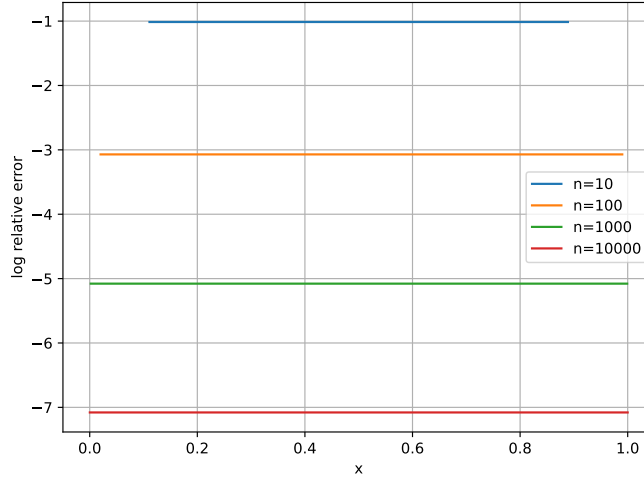


FIG. 4. Logarithm of the relative error representation (Eq. 13) for the numerical approximations v_i with respect to the exact solution $u(x)$, obtained for discretizations with $n = 10, 100, 1000$, and 10000 points.

Note that for a finer discretisation (larger n) the relative error is reduced. The error at the boundaries of the interval $[0, 1]$ tends to zero, as expected. The boundary values have not been included in Fig. 4.

When plotting the logarithm of the relative error for the four discretization cases, the possible maxima and minima are not clearly visible. Fig. 5 shows only the logarithm of the relative error for the case $n = 10000$, where the shape of the curve can be observed. In this case, the relative error varies within the order of 10^{-7} .

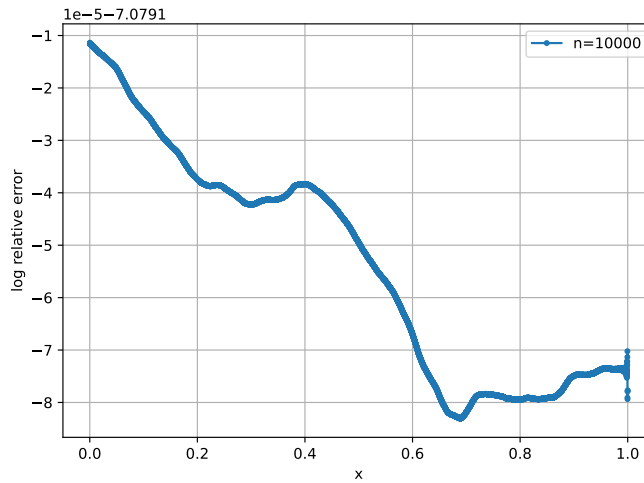


FIG. 5. Logarithm of the relative error representation (Eq. 13) for the numerical approximations v_i with respect to the exact solution $u(x)$, obtained for discretizations with $n = 10000$ points.

Problem c

Table I reports the maximum relative error for each discretisation case, ranging from $n = 10$ up to $n = 10^6$. In this part of Exercise 8, the programs `problem7.cpp` and `problem8.cpp` were slightly modified in order to extend the analysis to the cases ranging from $n = 10$ up to $n = 10^6$.

It is worth noting that, when discretising the function $u(x)$ and computing the approximations v using the Thomas algorithm, the behaviour of the relative error follows that of a second-order finite difference scheme. In particular, the error decreases proportionally to $O(h^2)$, with $h = 1/n$, as the discretisation is refined. Nevertheless, once n becomes sufficiently large, the error no longer decreases: it eventually stabilises due to the finite precision of floating-point arithmetic (around 10^{-15} for double precision). In practice, for n of the order 10^6 – 10^7 , the round-off error starts to dominate, and the relative error stops improving and may even increase. Thus, the minimum achievable error results from the balance between truncation error and rounding error. In this case, as seen in Table I, the relative error increases from the case $n = 10^5$ to the case $n = 10^6$.

TABLE I. Maximum relative error for the numerical solution v compared to the exact solution $u(x)$ different discretisation sizes n .

Number of points n	Output
10	9.6828×10^{-2}
10^2	8.4982×10^{-4}
10^3	8.3500×10^{-6}
10^4	8.3347×10^{-8}
10^5	1.4356×10^{-9}
10^6	8.4043×10^{-7}

PROBLEM 9

Problem a

For the signature $(-1, 2, -1)$ the tridiagonal matrix has $a_i = -1$, $b_i = 2$, and $c_{i-1} = -1$ for all interior indices. Starting from the general Thomas recurrences, $m = a_i/\tilde{b}_{i-1}$, $\tilde{b}_i = b_i - m c_{i-1}$, and $\tilde{g}_i = g_i - m \tilde{g}_{i-1}$, we obtain the simplified sweep

$$m = -\frac{1}{\tilde{b}_{i-1}}, \quad \tilde{b}_i = 2 - \frac{1}{\tilde{b}_{i-1}}, \quad \tilde{g}_i = g_i + \frac{\tilde{g}_{i-1}}{\tilde{b}_{i-1}},$$

followed by the back substitution

$$v_n = \frac{\tilde{g}_n}{\tilde{b}_n}, \quad v_i = \frac{\tilde{g}_i + v_{i+1}}{\tilde{b}_i} \quad (i = n-1, \dots, 1).$$

We refer to this as the *special algorithm*.

Algorithm 2 special algorithm

Input: system size n , right-hand side $\vec{g} = [g_1, \dots, g_n]$

Output: solution $\vec{v} = [v_1, \dots, v_n]$ to $A\vec{v} = \vec{g}$ with signature $(-1, 2, -1)$

Forward substitution

$\tilde{b}_1 \leftarrow 2, \quad \tilde{g}_1 \leftarrow g_1$

for $i = 2, \dots, n$ **do**

$$\tilde{b}_i \leftarrow 2 - \frac{1}{\tilde{b}_{i-1}}$$

$$\tilde{g}_i \leftarrow g_i + \frac{\tilde{g}_{i-1}}{\tilde{b}_{i-1}}$$

Back substitution

$$v_n \leftarrow \frac{\tilde{g}_n}{\tilde{b}_n}$$

for $i = n - 1, \dots, 1$ **do**

$$v_i \leftarrow \frac{\tilde{g}_i + v_{i+1}}{\tilde{b}_i}$$

Problem b

Using the specialised recurrences above, a minimal implementation performs the following floating-point operations (FLOPs):

Forward substitution: (For each $i = 2, \dots, n$)

$r_{i-1} = 1/\tilde{b}_{i-1}$ (1 division), $\tilde{b}_i = 2 - r_{i-1}$ (1 subtraction), and $\tilde{g}_i = g_i + r_{i-1}\tilde{g}_{i-1}$ (1 multiplication + 1 addition)

This is 4 FLOPs per step, for a total of $4(n - 1)$ FLOPs.

Back substitution:

First $v_n = \tilde{g}_n/\tilde{b}_n$ (1 division).

Then for each $i = n - 1, \dots, 1$, $v_i = (\tilde{g}_i + v_{i+1})/\tilde{b}_i$ (1 addition + 1 division), i.e. $2(n - 1)$ FLOPs, plus the initial division gives $2n - 1$ FLOPs in total.

Therefore, the specialised algorithm requires $\text{FLOPs} = 4(n - 1) + (2n - 1) = 6n - 5$.

For comparison, the general Thomas algorithm counted in Problem 6 requires $8n - 7$ FLOPs; the specialisation above thus reduces the cost by approximately $2n - 2$ FLOPs.

Problem c

The file `problem9.cpp` in the GitHub repository contains the implementation of the *special algorithm*. The code was executed for discretizations ranging from $n = 10$ up to $n = 10^6$, and the resulting data are stored in `data4.txt` within the same repository.

PROBLEM 10

To assess computational performance, timing tests were carried out for the *general algorithm* and the *special algorithm* over problem sizes ranging from $n = 10$ to $n = 10^6$. The timing instrumentation is implemented directly in the C++ sources `problem7.cpp` and `problem9.cpp`, respectively. Figure 6 was obtained from the Python script `plot4.py` and reports the wall-clock time required by each algorithm for every value of n .

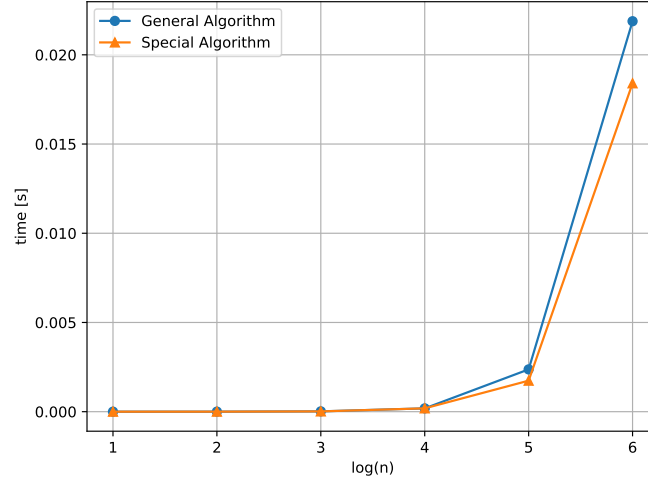


FIG. 6. Execution time versus problem size n (logarithmic scale) for the general and special algorithms.

The results indicate that the special algorithm achieves lower execution times compared to the general algorithm across all discretization sizes. The performance gap widens with increasing n , confirming the efficiency of the specialised formulation when the tridiagonal matrix has constant diagonals.